

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение

высшего образования

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет программной инженерии и компьютерной техники

Образовательная программа: Системное и прикладное программное
обеспечение

Направление подготовки: Программная инженерия 09.04.04

Дисциплина: Виртуальные машины

Практическое задание №1

Обучающийся: Пименов Глеб Андреевич 506758

Группа: Р4115

Дата 20.06.2026

Санкт-Петербург

2026

Оглавление

1. Цели	2
2. Задачи	3
3. Описание работы	4
4. Аспекты реализации	5
Результаты	6
Выводы	6

1. Цели

Используя комплекс программ, разработанный в первом семестре изучения дисциплины Системное программное обеспечение, реализовать формирование модели программы в терминах системы и модели памяти команд виртуальной машины по варианту. Полученную модель программы сериализовать и обеспечить выполнение программы посредством передачи сериализованного представления существующей реализации виртуальной машины.

2. Задачи

1. Изучить особенности виртуальной машины по варианту.
 - 1.1. Изучить соответствующие систему команд и модель памяти.
 - 1.2. Изучить существующие реализации данной ВМ и поддерживаемые ими форматы представления программ.
 - 1.3. Изучить внутреннюю организацию формата представления модели программ и особенности хода выполнения представленной таким образом программы.
2. Реализовать модуль, отвечающий за формирование модели программы в терминах целевой ВМ.

2.1. Описать необходимые структуры данных для представления модели программы в терминах целевой ВМ.

2.2. Программный интерфейс модуля принимает на вход структуру данных, содержащую графы потока управления и информацию о локальных переменных и сигнатурах для набора подпрограмм, а также информацию о необходимых пользовательских и встроенных типах данных.

2.3. В результате работы порождается структура данных, разработанная в п.2.1, в которой полученная в соответствии с п.2.2 информация представления в терминах целевой ВМ по мере необходимости.

3. Реализовать модуль, осуществляющий сериализацию сформированной модулем из п.2 структуры данных в выходной файл, организованный в соответствии с изученным форматом из п.1.3.

4. Реализовать основную программу, формирующую сериализованное представление в терминах целевой ВМ для подпрограмм, описанных файлах, имена которых переданы через аргументы командной строки.

4.1. Использовать модули, разработанные ранее в ходе изучения дисциплины Системное программное обеспечение для анализа входного текста.

4.2. Использовать модули, разработанные в п.2 и п.3 для формирования выходного файла, содержащего сериализованное представление программы.

5. Используя существующую реализацию ВМ, показать корректность формируемых выходных файлов и продемонстрировать выполнение тестовых программ.

6. Согласовать содержание отчёта с преподавателем, при необходимости внести изменения в разработанную программную систему и соответствующие правки в отчёт.

Вариант 2

Высокоуровневая машина - CLR (ECMA-335)

3. Описание работы

Программа `hlvm_lab1` предназначена для генерации CLR/CIL-кода из входных файлов на языке лабораторного комплекса. Она читает исходное представление программы, строит промежуточную модель и записывает результат в `.il` файл, готовый для сборки `ilasm`.

Внешний интерфейс:

```
./build_local/HLVM_LAB1_CLR <output.il> <input.txt>
```

Состав модулей:

`clr_model.h/.c` — формирует внутреннюю модель CLR-программы

`clr_serializer.h/.c` — сериализует модель в текст CIL

`main.c` — обрабатывает аргументы, вызывает парсеры, CFG, модель и запись `.il`

Способы использования:

собрать проект через CMake

сгенерировать `.il`

собрать `ilasm` и запустить результат в Mono

Примеры входной и выходной информации:

вход: `calc.txt` или `fib.txt`

выход: `calc.il`, `fib.il`

затем `ilasm /exe /output:hlvm_lab1/calc.exe hlvm_lab1/calc.il`

`mono hlvm_lab1/calc.exe`

Тесты:

проверка на fib.il с входом fib.txt

ожидаемый вывод: 3

4. Аспекты реализации

Внутренняя архитектура:

сначала используется существующий парсер и анализатор CFG

затем создаётся CLR-модель с методами, параметрами и локальными переменными

каждая ветка и переход представляются в виде CIL-инструкций

Особенности алгоритмов:

модель строится как линейная последовательность блоков CIL

обработка управления потоком выполняется через метки и переходы

сериализация учитывает синтаксис ECMA-335

Примеры кода:

clr_model.c содержит структуры для методов, аргументов, локальных переменных и инструкций

clr_serializer.c формирует секции .assembly, .method, .locals init и тела методов

main.c устанавливает путь к файлу, вызывает генерацию модели и запись .il

Результаты

Созданы артефакты:

исполняемый проект HLVM_LAB1_CLR

модули clr_model.*, clr_serializer.*, main.c

сгенерированные .il файлы calc.il, fib.il

Результаты тестов:

успешная генерация IL-кода

сборка через ilasm

запуск mono hlvm_lab1/fib.exe

Количественные оценки:

проверка выводов по тесту fib: ожидаемый результат 3

корректная генерация CIL для нескольких примеров входных файлов

Выводы

Цель задания достигнута: реализован конвейер от входного текста до CLR-совместимого .il кода.

Тесты показали, что программа генерирует работоспособный CIL и позволяет запускать результат через mono.

Это было достигнуто за счёт разделения работы на модель CLR и сериализацию, а также использования существующих компонентов парсера/CFG.

В процессе разработки подтвердилось понимание структуры CLR и особенностей генерации CIL, а также практики работы с промежуточными представлениями и сериализацией.